

LightCCI compare：gene 层 E 与 Sr 特征修复

Codex 中文 Markdown 计划对应 PDF / 不新增 gene h5ad / 不引入 spatial feature

代码版本	当前上传的 code.zip
核心目标	gene E 从 spot h5ad 派生并 PCA 到 64 维；gene Sr 用表达加权分配
明确边界	不把 spatial 写进 compare 向量矩阵，不使用 SR correlation，不要求 gene 级数据文件

本 PDF 是给研究记录和 Codex 执行使用的中文模板版，完整原文另附 Markdown。

LightCCI compare：gene 层 E 与 Sr 特征修复 Codex Plan

0. 本次任务目标

当前 `code.zip` 版本中，`light\_cci + compare\_E\_cos + gene:spot` 会失败：

```
ValueError: Found array with 0 feature(s) (shape=(15464, 0)) while a minimum of 1 is required by StandardScaler.
```

根因不是缺少 gene 级 h5ad。gene 层本来就是 LightCCI 里由 spot 层 GRN 虚拟出来的层，基因表达信息应从 spot 层 h5ad 的表达矩阵中派生，而不是去读取不存在的 `gene/heart/gene\_heart\*.h5ad`。

本次修改只解决两类 compare 特征：

1. `E`：gene 层表达特征从 spot 表达矩阵派生，按 “gene × spot 表达模式” 做 PCA，输出 64 维。
2. `Sr`：gene 层 Sr 特征从 spot-level Sr 通过 gene expression weighting 分配到 gene。

明确禁止：

- 不新增、不要求、不读取 gene 级 h5ad。
- 不把 spatial feature 加入 compare 的 E/Sr 向量矩阵。
- 不把 SR correlation 混进 E 特征。
- 不修改 `compare\_main\_lap\_sr\_spatial\_sot.py` 的主方法 spatial 逻辑。
- 不用 `context.lower\_mats / context.upper\_mats` 的 0 维 fallback 来掩盖 gene 层缺 h5ad 的问题。

1. 当前代码中需要关注的文件

主要修改文件：

```
mignet_ce/pij/compare/features.py
mignet_ce/io/developmental_features.py
scripts/profile_lightcci_compare_matrix.py
scripts/run_mignet_vertical.py
mignet_ce/config.py
```

辅助确认文件：

```
mignet_ce/networks/light_cci.py
mignet_ce/io/loaders.py
mignet_ce/representations/expression_only.py
mignet_ce/pij/compare/common.py
```

当前 `light\_cci.py` 中 gene graph 的设计是合理的：

```
if layer == "gene":
    grn_paths = resolver.paths("spot", organ, stage)
    return self._build_gene_graph(...)
```

问题主要出在 compare feature 构建阶段：`features.py::\_build\_expression\_base()` 和 `features.py::\_build\_sr\_base()` 又把 `gene` 当普通 layer 去找 h5ad / developmental feature CSV。

2. 设计原则

2.1 gene 层不是独立数据层

gene 层节点来自 spot 层 GRN 的 regulator/target gene 集合。

对于时间点 `t`，已有 spot 表达矩阵：

$$X_t: n_{\text{spot}}(t) \times n_{\text{gene}}$$

gene 层表达特征应使用：

$$X_t[:, \text{gene_units}].T: n_{\text{gene_nodes}}(t) \times n_{\text{spot}}(t)$$

即：

每一行 = 一个 gene 节点
 每一列 = 一个 spot
 值 = 该 gene 在该 spot 上的表达量

注意：不要因为某个 gene 在某些 spot 上表达量为 0 就删除这些 spot 列。列数不是“每个 gene 不一样”，而是所有 gene 都定义在同一批 spot 上，只是许多值为 0。

2.2 E 与 Sr 分离

`E` 只表达 gene 的表达模式，不包含 SR。

`Sr` 只表达 gene 被分配到的发育状态，不包含表达 PCA。

后续 `compare\_E\_Sr\_cos` 的语义才清楚：

E + Sr = 表达模式 + 发育状态分配

3. 修改一：gene 层 E 特征

3.1 新增 helper：判断 side 是否为 gene layer

在 `mignet\_ce/pij/compare/features.py` 增加：

```
def _is_gene_side(context: NetworkContext, side: str) -> bool:
    return _layer_for_side(context, side) == "gene"
```

3.2 新增 helper：从 spot h5ad 构造 gene × spot 矩阵

在 `features.py` 增加函数，建议命名：

```
def _build_gene_expression_pattern_mats(
    context: NetworkContext,
    cfg: TemporalRunConfig,
    side: str,
) -> tuple[list[np.ndarray], list[dict[str, object]]]:
    ...
```

行为要求：

1. 只允许在 `\_layer\_for\_side(context, side) == "gene"` 时调用。
2. 对每个 time point：
 - 用 `LayerDataResolver(cfg.data\_root).paths("spot", context.organ, stage)` 找 spot h5ad。
 - 读取 `read\_expression\_h5ad(spot\_paths.h5ad)`。
 - gene units 来自 `\_native\_units(context, side, idx)`。
 - 对每个 gene unit 从 `spot\_expr.expr.columns` 找对应 gene。
 - 生成矩阵 `M\_t = spot\_expr.expr.loc[:, present\_genes].T`，shape 为：

$$n_{\text{gene_nodes}}(t) \times n_{\text{spots}}(t)$$

3. 对 GRN 中存在但 h5ad 中不存在的 gene：

- 保留该 gene 的行。
- 行向量填 0。
- metadata 记录：`missing\_gene\_count`、`missing\_gene\_examples`。

4. 预处理：

- 对 gene 行做非负化：`np.maximum(values, 0)`。
- 建议对每个 gene 行做 row-sum normalization，使特征主要表达“表达分布模式”，避免高表达 gene 仅因总量大而支配 PCA。
- 然后按现有配置决定是否 `log1p`。
- 不要调用现有 `\_preprocess\_raw\_mats()` 直接处理，除非确认它的 row normalization 语义已经适合 gene $\times$ spot。建议新写一个 gene-specific preprocess helper，避免语义混淆。

建议新增：

```
def preprocess_gene_expression_patterns(
    matrices: Sequence[np.ndarray],
    normalize: bool,
    log1p: bool,
    scale_factor: float,
) -> list[np.ndarray]:
    ...
```

3.3 PCA 降到 64 维

用户指定：gene 表达特征用 PCA 压成 64 维。

建议新增配置项，避免硬编码：

```
compare_gene_expression_pca_components: int = 64
```

需要同步修改：

```
mignet_ce/config.py
scripts/run_mignet_vertical.py
scripts/run_mignet_vertical_ablation.py # 如果保留 ablation 入口，也同步
```

命令行参数建议：

```
--compare-gene-expression-pca-components 64
```

如果不想加新参数，也可以直接复用 `--pij-feature-components 64`。但更推荐新增 gene 专用参数，因为 `pij\_feature\_components` 同时影响其他特征逻辑，语义不够精确。

3.4 PCA 不能每个时间点独立后直接比较

不要简单地对每个时间点单独 PCA，然后直接用 cosine 比较。因为不同时间点的 PC 轴会旋转、翻转，坐标系不一致。

建议实现方式：

1. 对每个时间点的 `gene  $\times$  spot` 矩阵分别做 PCA / randomized PCA，得到：

```
Z_t: n_gene_nodes(t)  $\times$  64
```

2. 选择第一个时间点作为 reference。

3. 对后续时间点，用 shared gene rows 做 Orthogonal Procrustes 对齐：

```
R_t = argmin || Z_t[shared_genes] R - Z_ref[shared_genes] ||_F
Z_t_aligned = Z_t R_t
```

4. 对齐后再进入 compare feature set。

如果觉得 Procrustes 第一版太重，可以先做最小可运行版，但 metadata 必须写清楚：

```
gene_pca_temporal_alignment = "not_aligned_first_version"
```

更推荐直接做 Procrustes，因为 gene:gene 跨时间 PIJ 的 cost 需要跨时间可比的坐标轴。

实现可使用：

```
from scipy.linalg import orthogonal_procrustes
from sklearn.decomposition import PCA
```

PCA 参数建议：

```
PCA(n_components=actual_components, svd_solver="randomized", random_state=cfg.nmf_seed)
```

actual components 需要安全裁剪：

```
actual = min(requested_components, n_gene_nodes, n_spots)
```

如果 actual < 64，则右侧 zero-pad 到 64，保证维度固定。

3.5 在 `\_build\_expression\_base()` 中接入 gene side

重写 `\_build\_expression\_base()` 的 side 构建逻辑，不要使用现在的“只要一側 h5ad 缺失就整体 fallback 到 context mats”。

建议结构：

```
for side in ("lower", "upper"):
    if _is_gene_side(context, side):
        side_raw, side_meta = _build_gene_expression_pattern_mats(context, cfg, side)
        side_features, side_pca_meta =
            _reduce_gene_expression_patterns_with_temporal_alignment(...)
    else:
        side_raw, side_meta = _build_regular_expression_mats(context, cfg, side)
        side_features = existing_regular_E_pipeline(...)
```

第一版可以更保守：

- 如果 pair 不含 gene，完全走原来的 `\_build\_expression\_base()` 逻辑。
- 如果 pair 含 gene，走新的 mixed pipeline。

重点要求：

不能再因为 gene h5ad 缺失而使用 context.lower_mats/context.upper_mats 的 0-feature matrix。

metadata 中必须写清楚：

```
{
  "feature_key": "E",
  "feature_source": "expression",
  "gene_expression_source": "virtual_from_spot_h5ad",
  "gene_expression_representation": "gene_by_spot_pca",
  "gene_expression_pca_components": 64,
  "gene_expression_temporal_alignment": "orthogonal_procrustes_to_first_timepoint",
  "no_spatial_feature_used": true
}
```

4. 修改二：gene 层 Sr 特征

4.1 目标定义

对于 gene 节点 `g`，使用 spot-level Sr 通过 gene expression weighting 分配：

$$Sr_g = \sum_s X_{sg} * Sr_s / (\sum_s X_{sg} + \text{eps})$$

语义：

这个 gene 主要表达在哪些发育状态的 spot 中。

这不是 correlation，也不是 spatial feature。

4.2 在 `features.py::\_build\_sr\_base()` 增加 gene side 特判

当前 `\_build\_sr\_base()` 直接调用：

```
load_developmental_features_for_pij(context, cfg, idx, "lower")
load_developmental_features_for_pij(context, cfg, idx, "upper")
```

这会导致 gene 层去找：

```
developmental_features/gene/heart_11.5_features.csv
```

或者尝试把 spot feature 聚合到 domain。gene 层没有这种 map，所以会失败或语义错误。

新增 helper：

```
def _build_gene_sr_from_spot_expression(
    context: NetworkContext,
    cfg: TemporalRunConfig,
    side: str,
    time_index: int,
) -> tuple[np.ndarray, dict[str, object]]:
    ...
```

行为要求：

1. 只在 layer == "gene" 时调用。
2. 读取 spot h5ad：

```
spot_paths = LayerDataResolver(cfg.data_root).paths("spot", context.organ, stage)
spot_expr = read_expression_h5ad(spot_paths.h5ad)
```

3. 读取 spot-level developmental features：

```
load_developmental_features_for_layer(
    development_feature_root=cfg.development_feature_root,
    data_root=cfg.data_root,
    layer="spot",
    organ=context.organ,
    stage=stage,
    units=spot_units,
    aggregation=cfg.pij_feature_aggregation,
    missing_policy=cfg.pij_missing_feature_policy,
)
```

4. 选择 Sr 列：复用 `\_select\_sr\_column()`。

5. 对每个 gene unit：

```
weights = expression[:, gene]
weighted_sr = (weights @ sr_values) / (weights.sum() + eps)
```

6. 如果 gene 不在 spot h5ad 或表达总量为 0：

- 第一版建议填 spot-level Sr 均值，避免 NaN 导致后续标准化失败。
- metadata 记录：`missing\_gene\_count`、`zero\_expression\_gene\_count`、`fallback\_value = spot\_sr\_mean`。

输出 shape：

```
n_gene_nodes × 1
```

feature name：

```
Sr:expression_weighted_sr
```

metadata：

```
{
    "feature_key": "Sr",
}
```

```
"feature_source": "developmental_sr",
"gene_sr_source": "expression_weighted_spot_sr",
"formula": "sum_s X_sg * Sr_s / (sum_s X_sg + eps)",
"no_sr_correlation_used": true,
"no_spatial_feature_used": true
}
```

4.3 非 gene side 保持原逻辑

如果 side 不是 gene：

- spot：继续直接读 spot feature。
- domain：继续按现有 `spot\_domain\_map` 聚合。

不要因为 gene 逻辑影响 `spot:seurat\_k40` 和 `seurat\_k40:seurat\_k150` 已经跑通的部分。

5. 修改三：standardization 逻辑

当前 `\_standardize\_base()` 写法：

```
all_values = np.vstack([*lower, *upper])
```

这假设 lower 和 upper 的 feature 维度完全相同，而且语义也可以一起标准化。

对于 `gene:spot`，即使都压成 64 维，gene PCA 和 spot expression PCA 的语义也不同。更稳妥的做法是：

- pair 不含 gene：保持原来的 lower+upper 合并标准化。
- pair 含 gene：按 side 分别标准化。

建议改为：

```
def _standardize_base(
    lower: Sequence[np.ndarray],
    upper: Sequence[np.ndarray],
    *,
    side_specific: bool = False,
) -> ...:
    ...
```

在 `build\_compare\_feature\_set()` 中：

```
side_specific = context.pair.lower_layer == "gene" or context.pair.upper_layer == "gene"
lower_scaled, upper_scaled, meta = _standardize_base(base.lower, base.upper,
side_specific=side_specific)
```

metadata 写：

```
{
"standardization": "side_specific_zscore_for_gene_pair"
}
```

如果不这么改，`gene:spot` 仍然可能跑，但统计语义不干净。

6. 修改四：profile 脚本

文件：

```
scripts/profile_lightcci_compare_matrix.py
```

当前 `\_inspect\_expression(paths)` 对 gene layer 会显示 h5ad 不存在，容易误导。

需要改成：

- 当 layer == "gene" :
- 表达来源标记为 `virtual\_from\_spot\_h5ad`。
- 检查 `spot` h5ad 是否存在。
- 报告 spot units、spot genes、gene graph node count、gene nodes present in spot h5ad、missing gene count。
- 当 layer == "gene" 且检查 Sr :
- Sr 来源标记为 `expression\_weighted\_spot\_sr`。
- 不再检查 `developmental\_features/gene/\*.csv`。
- 检查 `developmental\_features/spot/\*.csv` 是否存在。

建议输出字段：

```
{
  "layer": "gene",
  "expression_input_mode": "virtual_from_spot_h5ad",
  "spot_h5ad_exists": true,
  "gene_expression_pca_components": 64,
  "sr_input_mode": "expression_weighted_spot_sr",
  "gene_feature_requires_gene_h5ad": false
}
```

7. 修改五：命令与验证

7.1 单独验证 gene:spot + E

```
python -u scripts/run_mignet_vertical.py \
--data-root "/home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory" \
--output-root "/home/jovyan/work/2026 Causality/output/debug_lightcci_gene_E" \
--network-method light_cci \
--pij-method compare_E_cos \
--organs heart \
--time-points 11.5 12.5 \
--level-pairs gene:spot \
--development-feature-root "/home/jovyan/work/2026 Causality/output/developmental_features" \
--compare-gene-expression-pca-components 64 \
--export-pij \
--export-pij-topk 0 \
--pij-archive-root "/home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory/pij_lightcci_compare_debug" \
--progress \
--max-workers 16
```

如果没有新增 `--compare-gene-expression-pca-components` 参数，则改用：

```
--pij-feature-components 64
```

7.2 单独验证 gene:spot + Sr

```
python -u scripts/run_mignet_vertical.py \
--data-root "/home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory" \
--output-root "/home/jovyan/work/2026 Causality/output/debug_lightcci_gene_Sr" \
--network-method light_cci \
--pij-method compare_Sr_cos \
--organs heart \
--time-points 11.5 12.5 \
--level-pairs gene:spot \
--development-feature-root "/home/jovyan/work/2026 Causality/output/developmental_features" \
--export-pij \
--export-pij-topk 0 \
--pij-archive-root "/home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory/pij_lightcci_compare_debug" \
--progress \
```



```
--max-workers 16
```

7.3 验证完整命令

```
python -u scripts/run_mignet_vertical.py \
--data-root "/home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory" \
--output-root "/home/jovyan/work/2026
Causality/output/mignet_vertical_lightcci_compare_seurat/network=light_cci/pij=compare_E_cos" \
--network-method light_cci \
--pij-method compare_E_cos \
--organs heart \
--time-points 11.5 12.5 13.5 14.5 \
--level-pairs gene:spot spot:seurat_k40 seurat_k40:seurat_k150 \
--development-feature-root "/home/jovyan/work/2026 Causality/output/developmental_features" \
--compare-gene-expression-pca-components 64 \
--export-pij \
--export-pij-topk 0 \
--pij-archive-root "/home/jovyan/public/datasets/Mouse-
embryo/E1S1_domain_factory/pij_lightcci_compare" \
--progress \
--max-workers 16
```

如果没有新增新参数，则同样改用：

```
--pij-feature-components 64
```

8. 必须通过的验收标准

1. `gene:spot + compare\_E\_cos` 不再报 `shape=(..., 0)` 的 StandardScaler 错误。
2. 代码不会尝试读取 `gene/heart/gene\_heart\_\*.h5ad`。
3. `feature\_source.json` 中 gene E 明确标记为：

```
virtual_from_spot_h5ad + gene_by_spot_pca + 64 components
```

4. `feature\_source.json` 中 gene Sr 明确标记为：

```
expression_weighted_spot_sr
```

5. `compare\_E\_cos`、`compare\_E\_Sr\_cos`、`compare\_Sr\_cos` 在 `gene:spot` 上都能跑完。
6. `spot:seurat\_k40` 和 `seurat\_k40:seurat\_k150` 结果不被破坏。
7. `run\_summary.csv` 中三组 pair 都应该是 `written`，不应再有 gene pair 的 error。
8. 输出的 `features\_source.npy` 对 gene side 应为：

```
n_gene_nodes × 64
```

9. 输出的 `features\_target.npy` 对 spot side 应正常存在，不能是 0 维。
10. 本次改动不新增 spatial feature，不修改主方法 spatial 逻辑。

9. Codex 执行顺序建议

1. 先在 `features.py` 增加 gene side 判断 helper。
2. 实现 gene expression pattern matrix 构建。
3. 实现 gene expression PCA 64 维，最好带 Procrustes 跨时间对齐。
4. 修改 `\_build\_expression\_base()`，让 gene side 走 virtual-from-spot 逻辑。

5. 实现 gene Sr expression-weighted 分配。
6. 修改 `\_build\_sr\_base()`，让 gene side 走 weighted Sr，非 gene side 走旧逻辑。
7. 修改 `\_standardize\_base()`，支持 gene pair side-specific z-score。
8. 修改 config 和 CLI，加入 `--compare-gene-expression-pca-components 64`；若不加新参数，至少确保 `--pjj-feature-components 64` 可以控制 gene E PCA。
9. 修改 profile 脚本，避免把 gene h5ad 缺失报告成错误。
10. 跑最小验证命令，再跑完整 compare_E_cos 命令。

10. 关键提醒

这次修复的中心不是“补 gene 数据文件”，而是承认 gene layer 是 virtual layer：

```
gene graph 来自 spot GRN
gene expression 来自 spot h5ad 的列
gene Sr 来自 spot Sr 的表达加权分配
```

所以代码应该围绕 spot 数据派生 gene 特征，而不是让用户额外制造 gene 级数据。